PHP OOP

مسار تعلّم تدريجي من أساسيات البرمجة كائنية التوجه إلى التجريد وتعدد الأشكال وإعادة استخدام الكود وتصميمه.

- **الأساسيات**
  - Class
    - القالب أو التصميم الذي يحدد شكل الكائن
    - لا يحتوي على بيانات كائن فعلية قبل إنشاء Object
    - مثال: `class User {}`
  - Object
    - نسخة فعلية من Class
    - يُنشأ باستخدام `new`
    - يمكن إنشاء عدة Objects من Class واحد
    - مثال: `$u = new User();`
  - Property
    - متغير داخل Class يمثل بيانات Object
    - مثال: `public string $name;`
  - Method
    - دالة داخل Class تمثل سلوك Object
    - مثال: `function getInfo() { ... }`
  - `$this`
    - يشير إلى Object الحالي
    - يستخدم للوصول إلى Properties وMethods
    - مثال: `$this->name`
  - Constructor
    - Method تعمل تلقائيًا عند تنفيذ `new`
    - تستخدم لتهيئة Properties
    - اسمها `__construct`
    - مثال: `function __construct($name) { $this->name = $name; }`
  - Constructor Promotion
    - اختصار متاح في PHP 8
    - ينشئ Property ويضع القيمة فيها في خطوة واحدة
    - مثال: `function __construct(private string $name) {}`
- **Encapsulation والتحقق**
  - Visibility
    - `public`: متاح من أي مكان
    - `private`: متاح داخل نفس Class فقط
    - `protected`: متاح داخل Class والأصناف التي ترث منه
  - Encapsulation
    - حماية البيانات وإخفاء تفاصيلها الداخلية
    - جعل Properties الحساسة `private`
    - التعامل مع البيانات من خلال Methods محددة
    - **يمنع القيم غير الصحيحة مثل الرصيد السالب**
  - Getter and Setter
    - Getter يعيد قيمة Property مثل `getBalance()`
    - Setter يغير قيمة Property بعد التحقق مثل `setAge()`
    - ليس كل Property يحتاج Getter وSetter
  - Validation
    - التحقق من القيمة قبل تخزينها
    - رفض القيمة غير الصحيحة بدل حفظها
    - يمكن استخدام Exception عند فشل التحقق
  - Exceptions
    - `throw` يرمي خطأ
    - مثال: `throw new InvalidArgumentException("msg");`
    - `try` يحتوي الكود المحتمل أن يسبب خطأ
    - `catch` يمسك الخطأ ويتعامل معه
    - `finally` ينفذ في كل الأحوال
  - Custom Exception
    - Class خاص يرث من `Exception`
    - يستخدم لخطأ محدد مثل `InsufficientFundsException`
- **Static والثوابت**
  - `static`
    - تتبع Class نفسه وليس Object معينًا
    - يمكن استخدامها دون إنشاء Object
    - مثال: `Logger::log("msg");`
  - `self::`
    - تشير إلى Class الحالي
    - مثال: `self::$count++`
  - `const`
    - قيمة ثابتة لا تتغير
    - مثال: `const PI = 3.14;`
    - الوصول إليها يكون مثل `Circle::PI`
  - `readonly`
    - Property يمكن تعيينها مرة واحدة فقط
    - متاحة منذ PHP 8.1
    - محاولة تغييرها بعد التعيين تسبب Error
  - Immutable Object
    - Object لا تتغير حالته بعد إنشائه
    - عند الحاجة إلى تغيير قيمة يتم إرجاع Object جديد
    - `readonly` يساعد في بناء هذا النوع من Objects
- **الوراثة**
  - Inheritance
    - إعادة استخدام وتوسيع كود Class آخر
    - Parent Class هو الصنف الأب
    - Child Class هو الصنف الابن
  - `extends`
    - ينشئ علاقة الوراثة
    - مثال: `class Employee extends Person`
    - الابن يرث الأعضاء المتاحة من الأب
  - `parent::`
    - يستدعي كودًا من Class الأب
    - مثال: `parent::__construct($name, $age);`
    - يمنع تكرار كود تهيئة الأب
  - Overriding
    - إعادة كتابة Method موروثة داخل الابن
    - نفس الاسم مع سلوك مختلف
    - يمكن استدعاء تنفيذ الأب باستخدام `parent::method()`
  - `final`
    - `final class` تمنع الوراثة منها
    - `final function` تمنع إعادة تعريفها
  - `instanceof`
    - يفحص نوع Object
    - مثال: `$x instanceof Admin`
  - قواعد الوراثة
    - لا يمكن للابن الوصول مباشرة إلى Private في الأب
    - يمكن للابن الوصول إلى Protected
    - يجب الحفاظ على توافق توقيع Method
- **التجريد وتعدد الأشكال**
  - Abstraction
    - إظهار المهم وإخفاء تفاصيل التنفيذ
    - يحدد ما يجب أن يفعله الكائن دون فرض طريقة التنفيذ
  - Abstract Class
    - لا يمكن إنشاء Object منها مباشرة
    - قد تحتوي على Abstract Methods غير مكتملة
    - الأبناء ملزمون بتنفيذ Abstract Methods
    - يمكن أن تحتوي أيضًا على Methods مكتملة
    - مناسبة لعلاقة `is-a` مثل Dog هو Animal
  - Interface
    - عقد يحدد Methods يجب تنفيذها
    - يستخدم مع `implements`
    - Class واحد يمكنه تنفيذ أكثر من Interface
    - مناسب لوصف قدرة مثل `Printable` أو `Payable`
  - الفرق بين Abstract Class وInterface
    - Abstract Class يمكن أن تحتوي Properties وحالة وكودًا مكتملًا
    - Class يرث عادةً من Abstract Class واحدة
    - Interface يركز على العقد والسلوك المطلوب
    - Class يمكنه تنفيذ عدة Interfaces
  - Polymorphism
    - نفس الاستدعاء يعطي سلوكًا مختلفًا حسب نوع Object
    - مثال: `$payment->pay(100)` مع Cash أو Card
    - يقلل استخدام `if` و`else` المتكررة
    - يسهل إضافة أنواع جديدة دون تغيير الكود الأساسي
    - يقلل الاعتمادية على Class محدد
  - Type Hinting
    - يحدد النوع الذي تقبله الدالة
    - مثال: `function pay(PaymentMethod $m)`
    - يجعل العقد بين الأجزاء أوضح
  - مثال Animal
    - `Animal` يحدد `makeSound()`
    - `Dog` ينفذها بصوت `Bark`
    - `Cat` ينفذها بصوت `Meow`
    - الكود يستدعي Method نفسها دون فحص النوع
- **إعادة استخدام الكود**
  - Trait
    - كود جاهز يمكن وضعه في أي Class باستخدام `use`
    - يعالج الحاجة إلى مشاركة كود بين Classes غير مرتبطة بالوراثة
    - مثال: Trait خاص بـ Timestamps
  - Composition
    - Object داخل Object آخر
    - تمثل علاقة `has-a`
    - مثال: Cart يحتوي Products
    - غالبًا أفضل من الوراثة عندما لا توجد علاقة `is-a`
  - Dependency Injection
    - Class يستقبل Dependencies من الخارج
    - لا ينشئ ما يحتاجه داخليًا باستخدام `new`
    - يسهل الاختبار والتبديل بين implementations
    - مثال: `new UserService(new SmtpMailer)`
  - Method Chaining
    - Method تعيد `$this`
    - تسمح باستدعاء عدة Methods في سطر واحد
    - مثال: `$qb->table("a")->where("b")->limit(5)`
- **PHP وواجهات Magic Methods**
  - Magic Methods
    - Methods بأسماء خاصة تبدأ وتنتهي بشرطتين سفليتين
    - تستدعيها PHP تلقائيًا في حالات محددة
  - `__construct`
    - تعمل عند إنشاء Object
  - `__toString`
    - تعمل عند طباعة Object باستخدام `echo`
  - `__get` و`__set`
    - تعملان عند قراءة أو كتابة Property غير موجودة أو غير متاحة
  - `__call`
    - تعمل عند استدعاء Method غير موجودة أو غير متاحة
  - `__isset`
    - تعمل عند استخدام `isset($obj->x)`
  - Countable
    - تجعل `count($obj)` تعمل على Object
  - ArrayAccess
    - تجعل الوصول مثل `$obj["key"]` ممكنًا
  - IteratorAggregate
    - تسمح باستخدام `foreach` على Object
  - JsonSerializable
    - تتحكم في الشكل الناتج من `json_encode($obj)`
  - Enum
    - مجموعة محدودة من القيم الثابتة
    - متاحة منذ PHP 8.1
    - مثال: `Pending` و`Paid` و`Shipped`
    - مثال: `OrderStatus::from("paid")`
- **Design Patterns ومبادئ التصميم**
  - Singleton
    - يضمن وجود Object واحد فقط في التطبيق
    - يستخدم Constructor خاصًا و`getInstance()`
  - Factory
    - Class مسؤول عن إنشاء Objects حسب النوع
    - مثال: `NotifierFactory::create("sms")`
  - Strategy
    - تبديل الخوارزمية باستخدام Object يطبق Interface
    - مثال: Percent Discount أو Fixed Discount
  - Observer
    - Subject يبلغ المشتركين عند حدوث Event
    - مثال: Newsletter وSubscribers
  - Repository
    - Class مسؤول عن حفظ واسترجاع البيانات
    - يمكن تبديله بين Memory أو File أو Database
    - بقية الكود لا يحتاج إلى معرفة مصدر البيانات
  - Value Object
    - Object يمثل قيمة مثل Money
    - تتم مقارنته بالقيمة لا بالهوية
    - غالبًا يكون Immutable
  - Single Responsibility
    - كل Class مسؤول عن مهمة واحدة فقط
    - افصل Validator وRepository وUser
  - Open / Closed
    - أضف سلوكًا جديدًا دون تعديل الكود القديم
    - يمكن استخدام Interface جديد لتحقيق ذلك
  - Program to Interface
    - اعتمد على Interface لا على Class محدد
    - يسهل تبديل التنفيذ والاختبار
  - Favor Composition
    - فضّل Composition على Inheritance عندما لا تكون علاقة `is-a` حقيقية
- **الأخطاء الشائعة والمراجعة**
  - نسيان `$this->`
    - يجعل الاسم متغيرًا محليًا بدل أن يكون Property
  - استخدام `new` مع Abstract Class أو Interface
    - غير مسموح في PHP
  - الوصول إلى Private من خارج Class
    - يسبب Fatal Error
    - استخدم Getter أو Business Method
  - نسيان `parent::__construct()`
    - يمنع تهيئة الجزء الموروث من الأب
  - محاولة تغيير `readonly`
    - تسبب Error بعد أول تعيين
  - ○ قائمة المراجعة
    - ☐ أميز بين Class وObject
    - ☐ أنشئ وأهيئ Objects باستخدام Constructor
    - ☐ أستخدم `$this` بشكل صحيح
    - ☐ أفرق بين Property وMethod
    - ☐ أطبق public وprivate وprotected
    - ☐ أستخدم Encapsulation وValidation
    - ☐ أفهم static وconst وreadonly
    - ☐ أطبق Inheritance وOverriding وparent::
    - ☐ أفرق بين Abstract Class وInterface
    - ☐ أستخدم Polymorphism وType Hinting
    - ☐ أختار Composition أو Inheritance بشكل صحيح
    - ☐ أفهم Traits وDependency Injection
    - ☐ أتعامل مع Magic Methods وواجهات PHP
    - ☐ أتعرف على Design Patterns الأساسية

Presented with xmind